

NORAYA

Project Handover Document

Telemedicine Backend Platform

Document Type	Technical & Operational Handover
Project Name	Noraya
Prepared By	Meenakshi Nanta
Role	Developer
Document Date	06 August 2026
Repository	NorayaTech / Back-End-Code
Classification	Internal — Confidential

This document provides a complete handover of the Noraya backend system, including architecture, modules, APIs, admin operations, integrations, scheduled jobs, and operational guidance for continuing development and support.

Table of Contents

1. Introduction & Purpose
2. Project Overview
3. Technology Stack
4. System Architecture & User Roles
5. Core Business Modules
6. Admin Panel Capabilities
7. API Reference Summary
8. Payment, Refund & Payout Flows
9. Appointment Lifecycle
10. Affiliate & Referral System
11. Notifications & Emails
12. Authentication & Security
13. Database Models
14. Scheduled Jobs
15. Project Structure
16. Environment & Configuration
17. Deployment & Operations
18. Handover Checklist

1. Introduction & Purpose

This handover document has been prepared by **Meenakshi Nanta**, Developer of the **Noraya** project, to transfer complete technical and operational knowledge of the system to the receiving team. It covers what the platform does, how it is built, how day-to-day features work, and what is required to run, maintain, and extend the product.

The intended audience includes backend developers, DevOps engineers, product owners, and administrators who will own ongoing delivery after handover.

2. Project Overview

Noraya is a telemedicine platform that connects patients with doctors for online consultations. This repository is the **Laravel backend** that powers:

- Doctor mobile / web application APIs
- Patient mobile application APIs
- Admin web panel (Blade + AdminLTE-style UI)
- Public legal & CMS pages (privacy, terms, refund policy, disclaimer, about, contact)
- Affiliate referral registration pages and QR-based referral flows
- Flutter / SPA fallback via `public/index.html`

Key product capabilities

- Doctor registration with credential verification and admin approval
- Patient discovery: search, favourites, ratings, public doctor profiles
- Appointment booking with symptoms, accept/reject, payment, and cancellation
- Push notifications via Firebase Cloud Messaging (FCM)
- Prescriptions and post-consultation patient reviews
- Razorpay payment verification, refunds, and monthly doctor payout ledger
- Affiliate referral links so patients can register via a referral code
- Master data management: countries, languages, specialities, settings

Production references observed in the codebase include domains such as **app.noraya.in** and **doctor.noraya.in**, and the patient Android package default `com.app.norayapatient`. Source control is hosted under **NorayaTech / Back-End-Code** (branch typically `dev`).

3. Technology Stack

Layer	Technology
Framework	Laravel 10 (PHP ^8.1)
Database	MySQL (Eloquent ORM)
Admin UI	Blade templates (AdminLTE-style)
Auth — Doctors	Laravel Passport (OAuth2 password grant)
Auth — Patients	Custom encrypted Bearer token middleware
Auth — Admin	Session guard against admins table
Payments	Razorpay (verify + refund APIs)
Push notifications	Firebase Cloud Messaging (HTTP v1 + Google service accounts)
Email	SMTP / Laravel Mailables

Layer	Technology
Error monitoring	Sentry (sentry/sentry-laravel)
HTTP client	Guzzle / Laravel HTTP
Timezone	Asia/Kolkata

4. System Architecture & User Roles

The backend serves three primary roles. Clients communicate over HTTPS REST APIs; the admin panel uses server-rendered Blade views. Patients may also register through an affiliate referral link (affiliate is a registration/attribution path, not a login role).

Role	Storage	Authentication	Primary surface
Admin	admins	Session (web guard)	/admin/* Blade panel
Doctor	users	Passport OAuth2 (auth:api)	Doctor app APIs
Patient	patients	AuthenticateToken middleware	Patient app APIs

Doctors must have `chrApproval = Y` before most doctor features are available.

5. Core Business Modules

5.1 Doctor lifecycle

- Register via POST `/api/register` (credentials, fees, speciality, languages, regulatory fields).
- Status remains pending until admin approval; registration email is sent.
- Admin approves → approval email (`SendApproval`).
- Doctor logs in via Passport password grant to `/oauth/token`.
- Manages availability, profile, bank details, appointment requests, and prescriptions.

5.2 Patient lifecycle

- Register via API or affiliate referral page (`PatientRegistrationService`).
- Login with email or phone; encrypted token stored in `patients.remember_token`.
- Browse / search doctors, favourite, fetch slots, book, pay, and submit feedback.

5.3 Supporting modules

- **Master data:** countries, states, languages, specialities (`dr_category`), general settings.
- **Favourites & ratings:** patient favourites and review ratings synced to doctor aggregates.
- **Doctor activity log:** admin-visible activity trail for doctors.
- **Book count:** booking counters reset by scheduled command based on settings.

6. Admin Panel Capabilities

Access: `/admin/login` → authenticated session. Sidebar navigation exposes the following modules:

Module	Path	Capabilities
Doctor	<code>/admin/doctor</code>	List approved doctors, bank details, payments summary, delete, activity log, record payout

Module	Path	Capabilities
Pending Approval	/admin/pending-doctor	Approve pending doctors and trigger approval email
Patient	/admin/patient	List and delete patients
Country	/admin/country	CRUD + publish flag
Language	/admin/language	CRUD + publish
Speciality	/admin/speciality	CRUD on doctor categories
Appointment	/admin/appointment	Paid non-canceled appointments; filter by date, doctor, speciality, country, state, search
Payment Logs	/admin/payment-log	Razorpay payments listing
Payment Ledger	/admin/payment-ledger	Monthly gross / commission / refunds / payout; mark month paid; per-doctor view
Affiliate	/admin/affiliate	CRUD affiliates, generate codes, default commission %, monthly/all-time stats, QR card
Settings	/admin/settings	Slot duration, admin commission %, affiliate commission %, reset book date
Profile	/admin/profile	Update admin profile

Admin can also initiate refunds via POST /admin/refund (same Razorpay refund handler used by the API).

7. API Reference Summary

All API routes are prefixed with /api. Postman collections exist under the postman/ directory.

7.1 Public endpoints

Method	Endpoint	Purpose
POST	/login	Doctor login → Passport tokens
POST	/register	Doctor registration
GET	/countries, /languages, /speciality, /state-list	Master data
POST	/set-localization-language	Set localization language
POST	/forget-password	Password reset email
GET	/public/doctor/{id}	Public doctor profile
POST	/refund	Refund by payment id (throttled)
POST	/patient-login	Patient login
POST	/patients-register	Patient registration (optional affiliate_code)

7.2 Doctor APIs (auth:api / Passport)

Home dashboard, appointment requests (all / today / next), profile update, availability, accept/reject patient request, patient profile & history, schedule, prescription, bank details upsert/fetch, logout.

7.3 Patient APIs (AuthenticateToken)

Home, view-all doctors, favourites, search, time slots, send booking request (throttle: booking), profile get/update, doctor profile, feedback, cancel appointment, clear conflict appointment, process payment (throttle: payment), my appointments, logout.

7.4 Web (non-API) routes of note

- Admin authentication and all /admin/* management pages
- Affiliate referral registration: GET/POST /refer/{code}
- Legal pages: privacy, terms (patient/doctor), refund policy, disclaimer, about, contact
- Password reset: password/reset/{token}
- SPA fallback: unmatched non-asset routes serve public/index.html

8. Payment, Refund & Payout Flows

8.1 Patient payment (Razorpay)

- Patient books an appointment (unpaid by default: charIsPaid = N).
- Client completes Razorpay checkout and sends payment id, doctor, appointment id, and amount.
- ProcessPayment verifies the payment via Razorpay API (status captured).
- Writes a row in payments and updates appointment paid flag.

8.2 Refunds

- Loads payment record, verifies Razorpay status, refunds remaining amount (paise-aware).
- Marks payment status as refunded.
- Available from API (throttle:payment) and admin panel.

8.3 Monthly doctor payout (admin ledger)

- Ledger aggregates successful payments per doctor for a selected month.
- Commission = gross × general_settings.percentage.
- Final payout = gross – commission – refunds.
- Doctor bank details shown from doctor_bank_details.
- Admin marks month as paid → updates payment monthly_payout flags and related logs.
- Optional artisan command payout:reset-monthly resets doctor monthly flags (not in default schedule).

9. Appointment Lifecycle

- Patient selects doctor and date → fetches time slots (default window ~09:00–21:00; duration from settings).
- Patient sends request with symptoms → appointment created (unpaid) → FCM to doctor.
- Doctor accepts or rejects → FCM to patient.
- Patient pays via Razorpay → appointment marked paid.
- Conflict handling: overlapping paid slots block new bookings; unpaid own requests show as requested.
- Meeting reminder push ~15 minutes before the appointment.
- Doctor adds prescription after consultation.
- Patient may cancel (reason + FCM).
- Review reminder push ~15 minutes after appointment end; patient submits feedback/rating.

10. Affiliate & Referral System

Affiliate is not a system login role. It is a referral mechanism that lets patients register through a unique code; admin manages affiliate records and commission tracking.

- Admin creates an affiliate record (custom person or linked doctor) with a unique referral code.
- Optional per-affiliate commission rate; otherwise default from settings (fallback ~3%).

- Patients register via `/refer/{code}` or by passing `affiliate_code` on API register.
- Referred patients store `affiliate_id` for attribution.
- Admin stats: referred users, paid non-canceled bookings, booking value, commission owed (month/year or all-time).
- Admin UI supports QR card export for sharing referral links.

11. Notifications & Emails

11.1 Push notifications (FCM)

Handled primarily via `NotificationController` using Firebase service account JSON files under `storage/app/firebase/` (doctor-app and patient-app). Examples: new appointment request, accept/reject, meeting reminder, review reminder.

11.2 Email notifications

Mailable	Trigger
DoctorRegistrationReceivedMail	Doctor registers
SendApproval	Admin approves doctor
ForgotPassword	Forget-password API (reset link)
UserOnboardingMail	Bank / onboarding reminder

Email Blade templates live under `resources/views/emails/`.

12. Authentication & Security

Actor	Mechanism	Notes
Doctor	Passport password grant	<code>PASSWORD_CLIENT_ID / SECRET</code> ; <code>auth:api</code> middleware
Patient	Custom encrypted Bearer	Payload contains id; must match <code>patients.remember_token</code>
Admin	Session (web → admins)	Standard form login / logout

Rate limiting

- `api`: 60 requests / minute (API group)
- `booking`: 10 / minute on patient send-request
- `payment`: 5 / minute on process-payment and refund

CORS is configured for doctor/patient web origins; CSP allows `iframe` embedding from `https://app.noraya.in` via `AllowIframeEmbedding`. Sentry is integrated for exception monitoring.

13. Database Models

Model	Table	Purpose
User	<code>users</code>	Doctors (fees, approval, FCM, languages, payout flags, etc.)
Patients	<code>patients</code>	Patients (phone, <code>affiliate_id</code> , token)
Admin	<code>admins</code>	Admin users

Model	Table	Purpose
Appointment	appointment	Bookings, status flags, prescription, amount
Payment	payments	Razorpay payment records + monthly_payout
PaymentLog	payment_logs	Payment ledger logs
RefundLog	refund_logs	Refund audit
DoctorCredit	doctor_credit	Accrued / paid doctor amounts
DoctorBankDetail	doctor_bank_details	Bank details for payouts
Affiliate	affiliates	Referral codes used when patients register
Rating / Favourite	ratings / favourite	Reviews and favourites
BookCount	book_count	Booking counters (cron reset)
GeneralSetting	general_settings	Key/value platform settings
Country / State / Language / DrCategory	country_master / states / language_master / dr_category	Master data
OrgExperience	org_experience	Doctor work history
DoctorActivity	doctor_activities	Admin activity trail

Note: Some core tables (for example appointment / patients) evolved earlier than the tracked migration set. For new environments, prefer a production DB dump plus running migrations, rather than migrations alone.

14. Scheduled Jobs

Command	Schedule	Purpose
meeting:reminder	Every minute	FCM 15 minutes before paid, non-canceled appointment
appointment:review-reminder	Every minute	FCM ~15 minutes after appointment end asking for review
reset:bookcount	Daily	If reset_book_date ≤ now, truncate book_count and clear setting

Optional (not in default schedule): payout:reset-monthly resets doctors' monthly_payout flags. Host should run `php artisan schedule:run` every minute.

15. Project Structure

```

noraya/
  app/
    Console/Commands/  # Cron / artisan commands
    Http/Controllers/  # API + web controllers
    Http/Controllers/Admin/
    Http/Middleware/   # AuthenticateToken, CSP, etc.
    Mail/              # Mailables
    Models/
    Services/          # PatientRegistration and related services
  config/              # auth, passport, services, sentry
  database/migrations/
  public/              # Web root, profiles, SPA index.html
  resources/views/admin/ # Admin UI
  resources/views/emails/

```

```
resources/views/referral/
routes/api.php, web.php
storage/app/firebase/ # Service account JSONs (required)
postman/              # API collections
staging/              # Staging-related folder (confirm on server)
```

Profile images are stored under `public/api/doctorprofile/` and `public/api/patientprofile/`.

16. Environment & Configuration

Configuration is driven by `.env`. Key variable groups (names only — never commit secrets):

- **App:** APP_NAME, APP_ENV, APP_KEY, APP_DEBUG, APP_URL
- **Database:** DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD
- **Cache / session:** CACHE_DRIVER, SESSION_DRIVER, REDIS_*
- **Passport:** PASSWORD_CLIENT_ID, PASSWORD_CLIENT_SECRET
- **Mail:** MAIL_MAILER, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM_*
- **Razorpay:** RAZORPAY_KEY, RAZORPAY_SECRET
- **Firebase / FCM:** FIREBASE_PROJECT_ID, FCM_SERVER_KEY + service account JSON files
- **AWS / Pusher / Sentry:** AWS_*, PUSHER_*, SENTRY_LARAVEL_DSN

17. Deployment & Operations

- Document root must point to `public/` (Apache/Nginx or XAMPP local).
- APP_URL must be correct — Passport token endpoint depends on it.
- Run `php artisan schedule:run` every minute (cron or Windows Task Scheduler).
- Ensure Firebase JSON credentials exist under `storage/app/firebase/`.
- Ensure Razorpay keys are set for payment verification and refunds.
- Composer install, migrate (or restore DB dump), storage link, optimize config/cache as needed.
- Git remote: GitHub **NorayaTech/Back-End-Code**; active development branch commonly `dev`.

Typical local setup (XAMPP)

- Place project under `htdocs`; serve via Apache with PHP 8.1+.
- Create MySQL database and configure `.env`.
- Run `composer install` and application key generation if needed.
- Import schema / run migrations; configure Passport clients for doctor login.

18. Handover Checklist

The receiving team should confirm possession of the following before assuming ownership:

#	Item	Status
1	Source code access (GitHub NorayaTech/Back-End-Code)	■
2	Production & staging server credentials / SSH / hosting panel	■
3	Database access + recent backup / dump	■
4	<code>.env</code> values for all environments (secure channel)	■
5	Razorpay dashboard access (keys, refunds)	■

#	Item	Status
6	Firebase projects + service account JSON files (doctor & patient)	■
7	Sentry project access	■
8	SMTP / mail provider credentials	■
9	Admin panel login credentials	■
10	Passport OAuth client id/secret	■
11	Cron / scheduler confirmation on servers	■
12	This handover document reviewed with stakeholders	■

Document Control

Prepared by: Meenakshi Nanta | **Role:** Developer | **Project:** Noraya | **Date:** 06 August 2026

This document reflects the state of the Noraya backend codebase at the time of writing. Subsequent releases should update this handover or append release notes as part of standard change management.

— End of Handover Document —